



ALVA'S INSTITUTE OF ENGINEERING & TECHNOLOGY

TECHNICAL PROJECT DOCUMENTATION

Patient Case-Taking Software using AI

Mangaluru Hospitals

Problem Statement ID 26047

Name	USN
K KHUSHI SHETTY	4AL23CS057
ANANYA	4AL23CS014

TABLE OF CONTENTS

1. Introduction.....	4
1.1 Background.....	4
1.2 Motivation.....	4
1.3 Scope of the Document.....	4
1.4 Organisation of the Document.....	4
2. Problem Statement and Objectives.....	5
2.1 Problem Statement.....	5
2.2 Problem Analysis.....	5
2.3 Objectives.....	6
2.4 Requirement Coverage.....	6
3. Analysis of Existing Systems.....	7
3.1 Existing Approaches.....	7
3.2 Gap Analysis.....	7
3.3 Proposed Approach.....	7
4. Requirements Analysis.....	8
4.1 User Roles.....	8
4.2 Functional Requirements.....	8
4.3 Non-Functional Requirements.....	9
4.4 Software and Hardware Requirements.....	9
5. System Architecture.....	10
5.1 Architectural Overview.....	10
5.2 Service Responsibilities.....	11
5.3 End-to-End Data Flow.....	11
5.4 Design Decisions.....	12
6. Technology Stack.....	13
6.1 Frontend.....	13
6.2 Backend.....	13
6.3 Database and Storage.....	14
6.4 AI/Voice Services.....	14
6.5 Deployment Tools.....	14
7. Data Collection and Case Ingestion.....	15
7.1 Sources and Collection Methods.....	15
7.2 Metadata Captured.....	15
7.3 Ingestion Steps.....	16
7.4 Handling Source Failure.....	16
8. Machine Learning and AI Assistant Module.....	17
8.1 Purpose.....	17
8.2 Follow-up Question Generation.....	17
8.3 Patient Detail Analysis.....	17
8.4 Recommendation and Allocation.....	18
8.5 AI Interaction Lifecycle.....	18
8.6 Evaluation Approach.....	18
9. Database Design.....	19
9.1 Overview.....	19
9.2 Core Tables.....	19
9.3 Relationships.....	19
9.4 Indexing Strategy.....	19
9.5 Migrations.....	20

10. API Design.....	20
10.1 REST Endpoints.....	20
10.2 Filter Parameters.....	20
10.3 AI Service Interface.....	21
10.4 Appointment Interface.....	21
10.5 Error Handling.....	21
11. Dashboard and User Interface Design.....	21
11.1 Views.....	21
11.2 Case Status Scale.....	21
11.3 Accessibility and Usability.....	22
11.4 Loading and Offline Behaviour.....	22
12. Admin Panel and Hospital Workflow.....	22
12.1 Purpose.....	22
12.2 Features.....	22
12.3 Case/Doctor Workflow.....	23
12.4 Feedback to the AI Module.....	23
13. Real-Time Appointment and Notification.....	23
13.1 Architecture.....	23
13.2 Reliability.....	23
14. Security and Privacy.....	24
15. Deployment.....	24
15.1 Local Deployment.....	24
15.2 Deployment Steps.....	24
15.3 Background Jobs.....	25
16. Testing Strategy.....	25
16.1 Levels of Testing.....	25
16.2 Sample Test Cases.....	25
17. Scalability and Performance.....	26
17.1 Current Scalability Features.....	26
17.2 Planned Scaling Path.....	26
18. Limitations and Future Scope.....	26
18.1 Limitations.....	26
18.2 Future Scope.....	26
19. Conclusion.....	27
References.....	27
Appendix A: Glossary.....	28
Appendix B: Configuration and Project Layout.....	28

1. Introduction

1.1 Background

Hospitals receive patient information through multiple channels: reception desks, phone calls, online forms, messaging applications and direct clinical consultations. When the initial information is incomplete, staff must repeatedly ask the same questions before a case can be routed to an appropriate doctor. This creates delays and makes it difficult to maintain a consistent record of the patient's symptoms, supporting evidence, preferred appointment time and contact details.

The Patient Case-Taking Software using AI is designed for Mangaluru hospitals as a guided digital intake platform. The system captures basic patient information, symptoms, relevant evidence and appointment requirements in a structured case. It adds a verification-code login mechanism for patients and separate role-based access for doctors and hospital authorities.

1.2 Motivation

The project is motivated by the need for a patient-friendly intake process that reduces unnecessary repetition while improving the quality of information available to doctors. An AI assistant can analyse the information already entered by the patient and ask focused follow-up questions instead of presenting a generic questionnaire. The system also considers practical hospital capacity: a doctor may be unavailable, on leave or already full, so the workflow must support automatic assignment to another eligible doctor.

1.3 Scope of the Document

This document describes the proposed and implemented aspects of the project, including patient verification, user roles, case intake, AI-assisted questioning, multilingual voice interaction, evidence handling, doctor allocation, appointment scheduling, hospital administration, database design, APIs, security, deployment, testing, scalability and future enhancements.

1.4 Organisation of the Document

Chapter	Content
1–3	Introduction, problem statement, objectives and analysis of existing systems
4	Functional and non-functional requirements
5–6	System architecture and technology stack
7–8	Case collection, AI assistant, recommendation and allocation
9–10	Database design and API design
11–13	Patient interface, hospital workflow and appointment/notification flow
14–17	Security, deployment, testing and scalability
18–19	Limitations, future scope, conclusion and references
Appendices	Glossary, configuration and project layout

Table 1.1: Document organisation

2. Problem Statement and Objectives

2.1 Problem Statement

Problem Statement ID 26047 – Patient Case-Taking Software using AI (Mangaluru Hospitals)

The objective is to design and develop a hospital-oriented patient case-taking platform capable of collecting, processing, analysing and securely storing patient information before clinical consultation. The system should provide a unified mechanism for recording patient details, symptoms, supporting evidence and appointment preferences while reducing repeated manual questioning.

The platform should support verified patient access using a verification code and separate authenticated workflows for doctors and hospital authorities. Patients should be able to create and manage their account, complete a case, interact with an AI assistant, select an available check-up date and time, and delete their account when required.

A major requirement is AI-assisted case analysis. The AI assistant should use the information already provided by the patient to identify missing or useful context and ask additional questions. The assistant is intended to support information collection and prioritisation; it is not a replacement for professional diagnosis or clinical judgement.

Doctor allocation must account for hospital capacity. If a doctor selected or initially matched for a case is already full, unavailable or on leave, the system should attempt to assign another eligible doctor rather than forcing the patient to repeat the entire process. The recommendation count should be configurable, with a default example of three suitable recommendations.

The final system should provide a traceable patient journey from verification and intake through AI questioning, evidence capture, doctor assignment and appointment booking, while maintaining role-based access and an audit record for important actions.

2.2 Problem Analysis

- Fragmented intake: patient information can be distributed across reception, phone, forms and consultation notes.
- Repeated questioning: incomplete initial information increases the time required before a case reaches the right doctor.
- Capacity mismatch: a doctor may be full, unavailable or on leave after the patient begins the process.
- Appointment uncertainty: patients may not have a clear view of available check-up times.
- Language barriers: a text-only interface may be less convenient for users who prefer Tulu, Kannada, English or Hindi.
- Limited structured assistance: generic forms do not adapt their questions to the information already supplied.
- Account-control requirement: patients need a clear way to access and delete their own account.
- Operational traceability: hospital authorities require visibility into doctors, cases, appointments and important actions.

2.3 Objectives

No.	Objective
1	Provide verification-code based patient login and protected patient access.
2	Capture patient details, symptoms, contact information, Mangaluru area and supporting evidence in a structured case.

No.	Objective
3	Use AI to analyse submitted details and ask relevant follow-up questions when information is incomplete.
4	Support multilingual voice interaction in Tulu, Kannada, English and Hindi.
5	Maintain doctor availability, leave and capacity information for allocation decisions.
6	Automatically assign another eligible doctor when the selected/initial doctor is full or unavailable.
7	Allow patients to select an available check-up date and time.
8	Provide configurable recommendations, with top-3 as a documented default example.
9	Provide separate doctor and hospital-authority workflows with role-based permissions.
10	Allow patients to manage and delete their account through the application.
11	Maintain evidence, appointment, verification and audit information for traceability.
12	Provide a modular architecture that can be extended to additional hospitals and services.

Table 2.1: Project objectives

2.4 Requirement Coverage

Problem statement requirement	How the proposed system addresses it
Patient identity and contact information	Patient details form and structured case record.
AI-assisted questioning	AI assistant analyses entered details and asks follow-up questions.
Doctor availability and capacity	Doctor roster, active status, leave and capacity are considered before assignment.
Automatic reassignment	Allocation checks other eligible doctors when the first doctor cannot accept the case.
Appointment scheduling	Available dates and time slots are presented for patient selection.
Multilingual voice	Voice interaction is planned/supported for Tulu, Kannada, English and Hindi.
Role-based access	Separate patient, doctor and hospital-authority workflows.
Account control	Patient can access and delete their account.
Traceability	Cases, evidence, appointments, verification events and audit actions are stored.

Table 2.2: Mapping of requirements to solution

3. Analysis of Existing Systems

3.1 Existing Approaches

Several approaches are commonly used for patient intake and appointment management. Each is useful for a particular purpose, but a hospital-specific case-taking workflow can improve continuity by connecting verification, AI-assisted questioning, doctor allocation and appointment selection.

System / Approach	Strengths	Limitations
Hospital reception / manual forms	Simple, familiar and immediately available	Repeated data entry; paper or disconnected records; difficult to scale.
Online appointment portals	Convenient scheduling and patient access	Often focus on booking rather than detailed case-taking and AI follow-up.
Telemedicine / consultation apps	Remote consultation and communication	Initial information may still be incomplete; hospital-specific capacity rules may differ.
Chatbot-based symptom forms	Interactive questioning	Generic questions may not use the full patient context or hospital roster.
Phone / messaging intake	Accessible and familiar	Unstructured data, difficult auditing and inconsistent appointment records.
Hospital information systems	Centralised clinical/administrative records	May require significant configuration and may not provide the proposed patient-facing AI workflow.

Table 3.1: Comparison of existing patient-intake approaches

3.2 Gap Analysis

The gap is not simply the absence of appointment software. The main gap is the absence of a single guided flow that combines verified patient access, structured case information, adaptive AI questioning, evidence capture, doctor-capacity checks, automatic reassignment, multilingual voice interaction and patient-selected appointment times.

3.3 Proposed Approach

The proposed approach follows six stages: Verify → Capture → Ask → Recommend → Allocate → Book. Verification establishes the patient session. Capture stores the initial case. The AI assistant asks focused questions when needed. The recommendation component produces a configurable set of suitable options. Allocation checks doctor capacity and can move to another eligible doctor. Finally, the patient selects an available date and time and the appointment is recorded.

Overall Approach

Verify → Patient Intake → AI Questions → Doctor Recommendation → Capacity Check → Reassignment if Required → Patient-selected Slot → Appointment

4. Requirements Analysis

4.1 User Roles

Role	Description
Patient	Registers/logs in through verification code, enters case details, responds to AI questions, uploads evidence where permitted, selects appointment time, views status and manages/deletes the account.
Doctor	Authenticates through a protected doctor workflow, views assigned cases, reviews patient-provided information and manages appointment/case status according to permissions.
Hospital Authority	Manages hospital configuration, doctor roster, capacity, availability, appointments, operational records and audit information.
AI Assistant	Analyses patient-provided information and generates relevant follow-up questions/recommendations. It does not replace clinical judgement.
System / Scheduler	Runs background checks such as slot availability, verification expiry, appointment status and other configured tasks.

Table 4.1: User roles

4.2 Functional Requirements

ID	Requirement
FR1	The system shall provide patient verification-code login.
FR2	The system shall support authenticated doctor and hospital-authority access.
FR3	The system shall capture patient name, age, gender, phone and Mangaluru area.
FR4	The system shall capture symptoms and case information.
FR5	The system shall allow evidence to be associated with a case.
FR6	The AI assistant shall analyse submitted patient details and ask follow-up questions.
FR7	The system shall support Tulu, Kannada, English and Hindi voice interaction where configured.
FR8	The system shall maintain doctor availability/capacity and leave information.
FR9	If a doctor is full/unavailable, the system shall attempt to allocate another eligible doctor.
FR10	The system shall provide a configurable number of recommendations.
FR11	The patient shall select an available check-up date and time.

ID	Requirement
FR12	The system shall create and track appointment status.
FR13	Patients shall be able to manage and delete their account.
FR14	Important actions shall be recorded in an audit trail.

Table 4.2: Functional requirements

4.3 Non-Functional Requirements

Category	Requirement
Security	Role-based access, protected sessions, verification-code handling, input validation and secure credential/secret management.
Privacy	Patient information must be accessible only to authorised users and should not be unnecessarily exposed in public views.
Availability	The core patient intake and appointment flow should remain usable when optional services fail.
Performance	Common patient and appointment operations should return within acceptable interactive response times on the deployment network.
Usability	The interface should use clear labels, readable typography, responsive layouts and understandable status messages.
Accessibility	Support keyboard-friendly controls, sufficient contrast, clear form labels and language/voice alternatives where available.
Maintainability	Use modular backend routes/services, database schema and isolated AI/voice components so features can be changed independently.
Scalability	Doctor, hospital, case and appointment volumes should be able to grow without redesigning the complete application.
Auditability	Important authentication, case, appointment and administrative actions should be traceable.

Table 4.3: Non-functional requirements

4.4 Software and Hardware Requirements

Item	Recommended / Project Requirement
Runtime	Python 3.x with Flask for the current backend workflow.
Database	SQLite/relational database for the current implementation; migration to PostgreSQL can be used for larger deployments.
Frontend	HTML templates, CSS and JavaScript; modern Chrome, Edge or Firefox.
AI layer	AI service/API or local model integration as configured; prompts

Item	Recommended / Project Requirement
	and outputs should be logged appropriately.
Voice layer	Speech recognition and text-to-speech services supporting Tulu, Kannada, English and Hindi where available.
Development machine	Dual/multi-core CPU, 8 GB RAM or more recommended, sufficient storage for the application and uploaded evidence.
Network	Internet access for hosted AI/voice services where applicable.
Deployment	Windows/Linux server or containerised Linux environment for production.

Table 4.4: Software and hardware requirements

5. System Architecture

5.1 Architectural Overview

The Patient Case-Taking Software uses a modular web architecture. The patient interface communicates with a Python/Flask application layer. The application manages authentication, case creation, evidence handling, doctor allocation and appointment operations. A relational database stores users, hospitals, doctors, leaves, cases, evidence, appointments, verification information and audit records.

The AI assistant is treated as a separate logical service/component so that the prompt, model provider or inference strategy can be changed without redesigning the complete patient workflow. A voice component provides speech input/output for supported languages. The scheduling component checks appointment slots and doctor capacity before confirming an appointment.

Layer	Components	Primary responsibility
Presentation	Patient portal, doctor portal, hospital authority portal	Forms, case views, appointment selection, status and role-specific controls.
Application	Flask routes/services	Authentication, case logic, allocation, appointment rules and permissions.
AI / Voice	AI assistant, speech-to-text, text-to-speech	Follow-up questions, structured analysis and multilingual voice interaction.
Data	Relational database, uploaded evidence	Persistent users, doctors, cases, evidence, appointments, OTPs and audit data.
Scheduling	Slot and capacity logic	Check availability, leave, capacity and alternate doctors.
Audit / Security	Verification, session control, audit logging	Protect access and provide traceability.

Figure 5.1: Layered architecture of the Patient Case-Taking Software

The screenshot shows the 'CareIntake Mangaluru' web application. At the top, a dark blue banner contains the text: 'In an emergency, go to the nearest hospital emergency department. This tool is not for emergencies.' Below this, the header includes a '+ CareIntake Mangaluru' logo and navigation links: 'New case', 'My appointments', 'Demo Patient', and 'Log out'. The main content area features four tabs: '1. Patient details' (active), '2. Symptoms and AI', '3. Evidence', and '4. Book a doctor'. The 'Patient details' form is titled 'Patient details' and includes a subtitle 'Basic information for the hospital record.' The form fields are: 'Full name' (text input with 'Demo Patient'), 'Age' (text input), 'Gender' (dropdown menu with 'Male' selected), 'Phone' (text input with '9999999999'), and 'Area in Mangaluru' (text input with 'e.g. Kadri, Bejai, Kankanady'). A 'Next: symptoms' button is at the bottom of the form.

Figure 5.2: Current patient-details interface from the supplied implementation

5.2 Service Responsibilities

Component	Technology / implementation	Responsibility
Patient Web Interface	HTML/CSS/JavaScript templates	Collect patient details, display AI interaction, evidence and appointment options.
Application Backend	Python / Flask	Authentication, cases, appointments, doctor routing and hospital workflows.
Database	SQLite / relational schema	Stores users, hospitals, doctors, leaves, cases, evidence, appointments, OTPs and audit events.
AI Assistant	AI model/service integration	Analyses case context and generates follow-up questions/recommendations.
Voice Component	Speech recognition + text-to-speech	Multilingual voice interaction for Tulu, Kannada, English and Hindi where supported.
Scheduler	Application/background tasks	Slot checks, expiry checks and configurable operational tasks.

Table 5.1: Components and responsibilities

5.3 End-to-End Data Flow

- Patient opens the website and authenticates using the verification-code login flow.
- Basic patient details and the initial case information are captured.
- Symptoms and optional evidence are added to the case.
- If the patient selects AI, the assistant analyses the information and asks follow-up questions.
- The completed case is considered for doctor recommendations according to configured rules.
- Doctor availability, leave and capacity are checked.
- If the first eligible doctor is full/unavailable, another eligible doctor is selected.
- The patient is shown available appointment dates and times and chooses a preferred slot.

- The appointment and case status are stored and made available to the authorised doctor and hospital authority.
- Important actions are written to audit records.

Figure 5.3: End-to-end patient-to-appointment processing flow

5.4 Design Decisions

- Flask-based application layer: keeps the current implementation lightweight and easy to maintain.
- Relational data model: supports clear relationships between patients, cases, doctors and appointments.
- Separate AI/voice boundary: allows model and language providers to change without rewriting the core workflow.
- Capacity-aware allocation: prevents a full doctor from becoming a dead end in the patient journey.
- Patient-selected slot: makes appointment preference explicit and reduces unnecessary rescheduling.
- Role separation: patient, doctor and hospital authority permissions are isolated.

5.4 Design Decisions

The architecture is intentionally modular because the project has three different categories of responsibility: patient-facing interaction, hospital operations and AI-assisted information processing. Keeping these responsibilities separated makes it possible to improve one part without changing the entire application.

Decision	Reason
Verification-code patient login	Provides a simple patient-facing authentication mechanism without requiring a traditional password at every first access.
Role-based doctor and authority access	Prevents patient users from accessing hospital management functions and keeps operational actions attributable.
Case-centric data model	Connects patient details, symptoms, evidence, AI questions and appointments through one case identifier.
Capacity-aware allocation	Ensures doctor assignment reflects real operational availability rather than only a static doctor list.
Automatic alternate doctor	Prevents a full/unavailable doctor from stopping the workflow.
Configurable recommendations	Allows hospital authorities to choose how many suggestions are presented; three can be the initial default.
Patient-controlled time selection	Lets the patient choose from actual available slots instead of relying on an unspecified appointment time.
Multilingual voice boundary	Makes Tulu, Kannada, English and Hindi support an extendable interface capability rather than hard-coding one language.
Audit records	Supports accountability for verification, case changes, appointments and administrative operations.

Safety principle: the AI assistant is an intake and information-support component. It should not independently diagnose a disease, prescribe medication or replace a qualified clinician. Clinical decisions remain with healthcare professionals.

6. Technology Stack

6.1 Frontend

Technology	Use in the project
HTML5	Structured patient, doctor and hospital-authority web pages.
CSS3	Responsive layout, forms, navigation, status indicators and accessibility-oriented styling.
JavaScript	Form interactions, AI/voice controls, dynamic appointment options and client-side validation where applicable.
Browser	Modern Chrome, Edge or Firefox for desktop and compatible mobile browsers.

Table 6.1: Frontend technologies

6.2 Backend

Component	Technology	Details
Web application	Python / Flask	Routes, sessions, authentication, case workflow, appointment operations and role checks.
Database access	SQLite / relational SQL	Structured persistence for hospital workflow entities.
File handling	Backend upload subsystem	Stores case-linked evidence and records metadata.
Authentication	Verification-code / OTP logic	Patient verification and role-based session handling.
AI integration	Python service/API boundary	Sends relevant case context for question generation and receives structured assistant responses.
Voice integration	Speech services	Speech-to-text and text-to-speech for supported languages.

Table 6.2: Backend components and libraries

The current project screenshot and terminal show a Python/Flask application being run locally with a relational schema. The architecture described here distinguishes implemented elements from planned production extensions.

6.3 Database and Storage

Component	Technology	Description
Relational database	SQLite / SQL	Stores hospital, user, doctor, leave, case, evidence, appointment, OTP and audit entities.
Uploaded evidence	Local uploads directory / configured	Case-linked files with filename and

Component	Technology	Description
	storage	method/metadata.
Indexing	SQL indexes	Supports appointment, doctor, case and lookup operations as data volume grows.
Audit storage	Audit table	Stores timestamp, user, action and detail for important operations.

Table 6.3: Data layer

6.4 AI/Voice Services

Service	Data / use
AI assistant	Patient-provided details, symptoms and previous answers; used to generate relevant follow-up questions and structured recommendations.
Speech-to-text	Converts patient voice input into text for AI/case processing.
Text-to-speech	Reads questions/status information back to the patient in a supported language.
Language selection	Tulu, Kannada, English and Hindi are exposed as patient-facing language options where the configured provider supports them.

Table 6.4: AI and voice components

6.5 Deployment Tools

The current project can be developed and tested locally using Python, Flask and the relational database shown in the supplied schema. For production, the same application can be placed behind Nginx or another reverse proxy, with environment variables for secrets, HTTPS, scheduled backups and a production database such as PostgreSQL.

7. Data Collection and Case Ingestion

7.1 Sources and Collection Methods

Unlike a weather-ingestion system, the primary source in this project is the patient. Additional information may be provided by the hospital workflow, doctor review, uploaded evidence and AI-generated follow-up answers. The system therefore focuses on controlled collection rather than uncontrolled web scraping.

Source	Collection method	Use
Patient	Verified web form / voice input	Identity, contact details, symptoms, area and case information.
Patient evidence	Upload interface	Documents/images/files associated with a case where permitted.
AI assistant	Interactive questions	Fills important information gaps identified from the initial case.

Source	Collection method	Use
Doctor	Protected portal	Clinical review, case status and appointment-related actions according to permissions.
Hospital authority	Administrative portal	Doctor roster, availability, capacity, appointments and configuration.
Verification service	Verification-code workflow	Confirms patient access before sensitive case information is shown.

Table 7.1: Sources and collection methods

7.2 Metadata Captured

Data group	Examples
Patient	Name, age, gender, phone, account/session identifiers.
Location	Area in Mangaluru, and optional location information where explicitly collected.
Case	Symptoms, description, AI answers, case status and timestamps.
Evidence	Kind, method, filename/storage reference and case association.
Appointment	Doctor, date, time slot, status and booking timestamps.
Doctor	Hospital, department, active status, capacity, leave and availability.
Security	Verification records, expiry, attempt count and audit information.

Table 7.2: Patient case metadata

7.3 Ingestion Steps

- Start a verified patient session.
- Create or load the patient case record.
- Validate required fields such as name, age, gender, phone and area.
- Store the initial symptoms and patient-provided description.
- Accept optional evidence and associate it with the case.
- Send only the necessary context to the AI assistant when the patient selects AI.
- Store the assistant questions and patient answers as part of the case workflow.
- Run recommendation/allocation rules using doctor availability and capacity.
- Present available appointment dates and time slots.
- Create the appointment after the patient confirms the selected slot.
- Record important actions in the audit trail.

7.4 Handling Source Failure

Failure	System behaviour
Verification service unavailable	Do not expose protected patient information; show a safe retry

Failure	System behaviour
	message.
AI service unavailable	Allow the patient to continue with the non-AI case flow where permitted, or clearly indicate that AI assistance is temporarily unavailable.
Voice service unavailable	Fall back to text interaction.
Evidence upload failure	Keep the case usable and report the upload failure without silently discarding the case.
No doctor available	Show that no eligible slot is currently available and allow retry/alternate time or hospital configuration.
Selected doctor becomes full	Re-run allocation and offer another eligible doctor rather than restarting the complete intake.
Database failure	Avoid confirming an appointment unless the transaction is successfully stored.

Table 7.3: Failure handling

The key design rule is graceful degradation: optional AI and voice features should not make the entire patient intake unusable when those services are temporarily unavailable.

8. Machine Learning and AI Assistant Module

8.1 Purpose

The AI module supports the collection and organisation of patient information. Its first responsibility is to understand what has already been entered and determine which additional questions may improve the completeness of the case. The assistant should avoid asking for information that is already present unless confirmation is necessary.

8.2 Follow-up Question Generation

A question-generation pipeline can be implemented using a structured prompt containing the case fields and previous answers. The AI response should be constrained to a small number of questions at a time, prioritising information that is relevant to routing or appointment preparation. Questions should use simple language and follow the patient's selected language where the configured voice/text service supports it.

Input	Processing	Output
Patient details	Validate required fields and normalise values	Structured patient context
Symptoms / description	Extract key terms and missing context	Potential information gaps
Previous answers	Avoid repeated questions	Next relevant questions
Language preference	Select response/voice locale	Translated/localised question
Case status	Determine whether more intake is useful	Continue / complete suggestion

Table 8.1: AI follow-up question pipeline

8.3 Patient Detail Analysis

- Check whether the initial description is specific enough for the next workflow stage.
- Identify missing context such as duration, severity-related descriptors, relevant associated symptoms or other non-diagnostic intake information configured by the hospital.
- Use previous answers to prevent repetitive questions.
- Return structured information that can be stored with the case.
- Keep a clear distinction between AI-generated intake assistance and clinical diagnosis.

8.4 Recommendation and Allocation

The recommendation component can produce a configurable number of suitable doctors or appointment options. The documented initial default is three recommendations, but the hospital authority can change this configuration. Recommendations should be generated from operational data such as doctor availability, active status, department or configured specialty, leave and current capacity.

Step	Rule
1	Collect eligible active doctors associated with the relevant hospital.
2	Remove doctors who are on leave or otherwise unavailable for the requested time.
3	Check current capacity for the relevant appointment window.
4	Rank remaining candidates using configured hospital rules.
5	Return the configured number of recommendations; default example: 3.
6	If the selected doctor becomes full/unavailable, re-run the allocation and assign another eligible doctor.
7	Confirm the appointment only after a valid date/time slot is selected and stored.

Table 8.2: Doctor recommendation and allocation logic

8.5 AI Interaction Lifecycle

Start → Read patient context → Identify missing information → Ask question → Receive answer → Update case context → Re-evaluate → Complete intake or ask another question → Recommend/route → Doctor review

8.6 Evaluation Approach

Area	Evaluation method
Question relevance	Review whether generated questions relate to missing case information.
Redundancy	Measure repeated questions against already captured fields and previous answers.
Language quality	Human review by speakers/users of Tulu, Kannada, English and

Area	Evaluation method
	Hindi where applicable.
Safety	Verify that responses do not present unsupported diagnosis or treatment as fact.
Allocation accuracy	Test doctor availability, leave and capacity scenarios including automatic reassignment.
Recommendation usefulness	Review whether the configured recommendation list contains eligible doctors/slots.

Table 8.3: AI and allocation evaluation approach

9. Database Design

9.1 Overview

The supplied project schema already demonstrates a relational structure for hospitals, users, doctors, leaves, cases, evidence, appointments, verification records and audit activity. The data model is case-centred: a patient case can link to evidence and appointments, while appointments link the patient/case to a doctor.

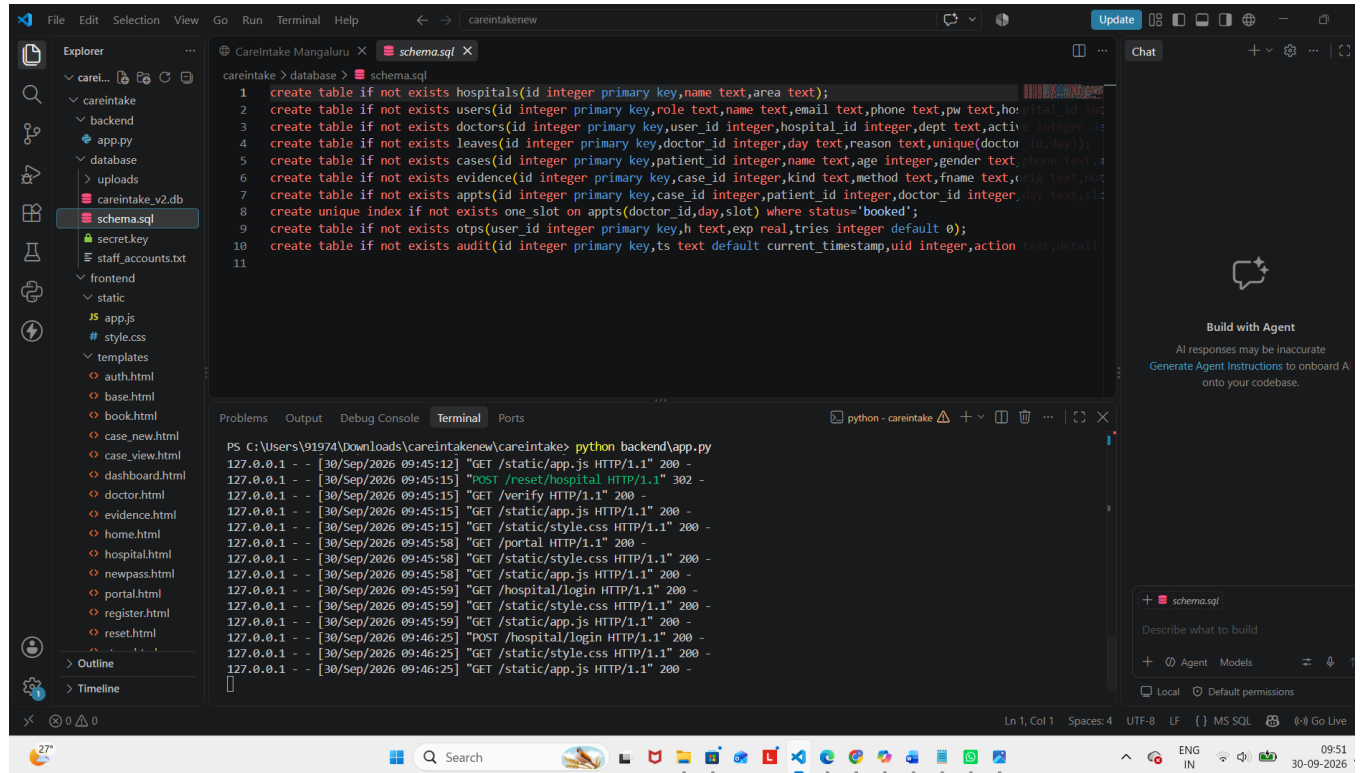


Figure 9.1: Supplied relational schema used as the project database foundation

9.2 Core Tables

Table	Purpose
hospitals	Hospital identity and Mangaluru-area information.
users	Login identity, role, contact information and hospital

Table	Purpose
	association.
doctors	Doctor-specific information linked to a user and hospital.
leaves	Doctor leave dates and reasons.
cases	Patient case details including identity/contact and intake information.
evidence	Case-linked uploaded evidence and its metadata.
appts	Appointment records connecting case, patient and doctor with date/time/status.
otps	Verification information, expiry and attempt/trial metadata.
audit	Timestamped user actions and details for traceability.

Table 9.1: Core tables

9.3 Relationships

- Users can represent patients, doctors or hospital authorities according to the role model.
- Doctors reference a user and hospital; leave records reference a doctor.
- Cases contain patient-facing intake information and can be associated with evidence.
- Appointments reference a case/patient and a doctor, with date, time and status.
- OTP records support verification-code login and attempt/expiry controls.
- Audit records reference the actor and store the action and detail.

9.4 Indexing Strategy

Index / key	Use
Primary keys	Fast identity and relationship lookups.
Case ID	Efficient retrieval of all evidence and appointments for a case.
Doctor + date/time	Appointment availability and capacity checks.
Patient + case status	Patient history and active-case lookup.
OTP expiry / user	Verification-code validation and cleanup.
Audit timestamp / user	Administrative and security investigation.
Hospital / doctor active status	Fast eligible-doctor selection.

Table 9.2: Indexing strategy

9.5 Migrations

Database migrations should be versioned so that a development database can be recreated consistently and future fields—such as appointment capacity, language preference, AI question history or hospital configuration—can be added without manually editing production records.

10. API Design

10.1 REST Endpoints

Endpoint	Method	Purpose
/verify	POST	Submit/validate patient verification information.
/case	POST	Create or update a patient case.
/case/<id>	GET	Retrieve an authorised case.
/evidence	POST	Upload/associate evidence with a case.
/ai/analyse	POST	Analyse case details and return follow-up questions.
/doctors/recommend	GET/POST	Return eligible doctor recommendations.
/appointments/slots	GET	Return available date/time slots.
/appointments	POST	Create an appointment after patient confirmation.
/doctor/cases	GET	Return cases assigned to the authenticated doctor.
/admin/doctors	GET/POST	Hospital-authority doctor roster operations.

Table 10.1: Primary API endpoints

10.2 Filter Parameters

Parameter	Meaning
date	Requested appointment date.
time_from / time_to	Preferred appointment time window.
hospital_id	Hospital context for doctor/slot selection.
doctor_id	Specific doctor where a patient or authority selects one.
status	Case or appointment status filter.
language	Preferred text/voice language: Tulu, Kannada, English or Hindi.
limit	Number of recommendations/results to return; configurable by hospital.

Table 10.2: Common filter parameters

10.3 AI Service Interface

The AI interface should receive only the case information required for the requested task. A structured request can contain patient context, symptoms, previous answers, selected language and the desired

operation. The response should contain questions or recommendations in a predictable structure so the web application can validate and display them safely.

10.4 Appointment Interface

The appointment interface should expose available slots rather than accepting arbitrary times. When the patient selects a slot, the backend should re-check availability before committing the appointment. This prevents two patients from being confirmed for the same capacity-constrained slot.

10.5 Error Handling

- 400: invalid or incomplete input.
- 401: missing/invalid authentication or verification state.
- 403: authenticated user lacks permission for the requested operation.
- 404: requested case, doctor, appointment or resource does not exist.
- 409: appointment slot or doctor capacity conflict; re-run availability and allocation.
- 422: valid request format but business validation failed.
- 500: unexpected server error; do not expose sensitive implementation details.

11. Dashboard and User Interface Design

11.1 Views

View	Purpose
Patient login / verification	Enter verification code and access protected patient workflow.
Patient details	Capture name, age, gender, phone and Mangaluru area.
Symptoms and AI	Enter symptoms and optionally invoke the AI assistant for follow-up questions.
Evidence	Upload or reference supporting evidence for the case.
Doctor recommendation	Show eligible doctor/recommendation options according to hospital rules.
Appointment booking	Display available date/time slots for patient selection.
Patient appointments	View booked appointments and their status.
Doctor portal	View assigned cases and appointment information.
Hospital authority portal	Manage doctors, capacity, availability, cases and operational settings.

Table 11.1: Application views

11.2 Case Status Scale

Status	Meaning
Draft	Patient has started a case but has not completed required intake.
AI Review	AI questions are being answered or the case is being analysed.
Ready for Allocation	Required intake information is available for doctor routing.

Status	Meaning
Assigned	A doctor has been allocated.
Booked	An appointment slot has been selected and confirmed.
Completed	The appointment/case workflow is completed according to hospital rules.
Cancelled	Appointment or case was cancelled.

Table 11.2: Case/appointment status model

11.3 Accessibility and Usability

- Use clear field labels and validation messages.
- Keep primary actions visible and consistent across steps.
- Provide language selection before voice interaction.
- Allow text fallback when voice services are unavailable.
- Use readable typography, sufficient contrast and keyboard-friendly controls.
- Make the AI assistant's role clear so patients do not interpret it as a doctor.

12. Admin Panel and Hospital Workflow

12.1 Purpose

The hospital-authority workflow provides operational control over doctors, availability, capacity, appointments and configuration. It is separate from the patient workflow so that administrative actions require an authorised role.

12.2 Features

Feature	Description
Hospital management	Maintain hospital identity and Mangaluru-area information.
Doctor roster	Add/manage active doctors and their hospital association.
Capacity settings	Configure the number of patients/appointments a doctor can accept in a period.
Leave management	Record leave/unavailability so the allocation engine does not recommend an unavailable doctor.
Recommendation count	Configure the number of doctor recommendations; initial documented default is 3.
Appointment monitoring	Review scheduled, pending, completed or cancelled appointments.
Case oversight	Review case status and route operational issues without changing clinical judgement.
Audit view	Review important administrative and authentication actions.

Table 12.1: Hospital-authority features

12.3 Case/Doctor Workflow

- Patient completes verified intake.
- System evaluates case completeness and doctor eligibility.
- Recommendations are produced using active status, availability, leave and capacity.
- Patient selects or is assigned an eligible doctor according to the configured workflow.
- If the doctor becomes full/unavailable, the allocation logic selects another eligible doctor.
- Hospital authority can update roster/capacity rules; changes should affect future allocation decisions.
- Doctor reviews the assigned case through the protected doctor portal.

12.4 Feedback to the AI Module

Where appropriate, hospital-authority and doctor review outcomes can be used as labelled feedback for improving question quality and recommendation rules. Any such training process must follow the hospital's privacy, consent, retention and governance requirements.

13. Real-Time Appointment and Notification

13.1 Architecture

Real-time behaviour is primarily important for appointment availability and operational status. When a patient requests available slots, the application reads the current schedule and doctor capacity. At booking time, the server re-checks the slot to avoid stale availability. Optional notification services can inform patients about booking confirmation, changes or reminders.

Step	Description
Request slots	Patient requests a date/time range.
Read availability	System checks appointments, capacity, leave and active doctor status.
Display	Available slots are shown to the patient.
Patient selection	Patient chooses a preferred time.
Final validation	Backend re-checks the selected slot immediately before confirmation.
Commit	Appointment is stored with doctor, date, time and status.
Notify	Optional SMS/email/WhatsApp/voice notification can be triggered where approved.

Table 13.1: Real-time appointment delivery steps

13.2 Reliability

- Availability is re-checked at booking time to reduce double-booking.
- Doctor leave and capacity changes invalidate incompatible slots.
- If notification delivery fails, the appointment record remains the authoritative status.
- If voice/AI services fail, text-based intake remains available where permitted.
- Client refresh/retry should retrieve the latest appointment state from the server.

14. Security and Privacy

Area	Measure
Authentication	Verification code for patients; authenticated sessions for doctors and hospital authorities.
Authorisation	Role checks restrict patient, doctor and authority operations.
Transport	HTTPS/TLS should be used in production.
Secrets	API keys and database credentials should be stored in environment variables or a secret manager.
Input validation	Validate form values, uploaded file types, identifiers and appointment requests.
Privacy	Patient details should not be exposed in public views or to unauthorised roles.
Auditability	Important actions should record actor, time, action and relevant detail.
Account deletion	Patient deletion should remove or anonymise data according to the hospital retention policy.

Table 14.1: Security and privacy measures

15. Deployment

15.1 Local Deployment

Component	Role
Python / Flask	Application server and business logic.
Relational database	Persistent project data.
Uploads directory	Case evidence storage during development.
Browser	Patient, doctor and authority interface.
AI/voice provider	Optional external service for AI and speech features.

Table 15.1: Current deployment components

15.2 Deployment Steps

- Install Python and project dependencies.
- Create/configure the application database and initialise the schema.
- Configure environment variables and secrets without committing them to source control.
- Configure the upload directory and appropriate file permissions.
- Start the Flask application in development mode for testing.
- For production, use a WSGI server behind Nginx/HTTPS and a production-grade database.
- Create regular database and evidence backups.
- Verify patient, doctor and authority login paths before opening the system to users.

15.3 Background Jobs

Background jobs can be used for verification-code expiry cleanup, appointment reminders, appointment status updates, doctor capacity synchronisation, audit maintenance and other scheduled operational tasks.

16. Testing Strategy

16.1 Levels of Testing

Level	Scope
Unit testing	Validation rules, recommendation count, capacity checks, slot calculations and account operations.
Integration testing	Flask routes with database, evidence storage, AI/voice adapters and authentication.
API testing	Authentication, case, AI, doctor recommendation and appointment endpoints.
UI testing	Forms, step navigation, AI interaction, language selection and appointment selection.
Security testing	Role boundaries, verification-code handling, session expiry and invalid input.
AI evaluation	Question relevance, repetition, language quality and safe output.
Performance testing	Appointment availability queries and doctor allocation under increasing load.

Table 16.1: Testing levels

16.2 Sample Test Cases

ID	Test	Expected result
T1	Enter valid verification code	Patient is authenticated and protected workflow opens.
T2	Enter invalid/expired code	Access is rejected and an appropriate retry message is shown.
T3	Submit incomplete patient details	Validation identifies required fields.
T4	Press AI after entering symptoms	Assistant analyses context and asks relevant follow-up questions.
T5	Select Kannada/Tulu/English/Hindi voice	Configured language/voice path is selected.
T6	Selected doctor is full	Another eligible doctor is recommended/assigned.
T7	Doctor is on leave	Doctor is excluded from the available recommendation list.
T8	Patient selects an available slot	Appointment is stored with the selected

ID	Test	Expected result
		date/time.
T9	Slot becomes unavailable before confirmation	Booking is rejected/rechecked and new slots are shown.
T10	Patient deletes account	Account deletion/anonymisation follows the configured retention policy.
T11	Patient calls doctor-only endpoint	Request is rejected by role authorization.
T12	AI service is unavailable	Text-only workflow remains usable or a safe fallback is shown.

Table 16.2: Sample test cases

17. Scalability and Performance

17.1 Current Scalability Features

- Modular Flask routes/services allow individual functions to be improved independently.
- Relational keys and indexes support predictable case and appointment lookups.
- Doctor allocation can operate on a filtered eligible set rather than scanning every historical appointment.
- Optional AI and voice services are isolated from the core case database.
- Patient/doctor/authority workflows are separated by role, reducing unnecessary data access.

17.2 Planned Scaling Path

Component	Role at higher volume
PostgreSQL	Production relational storage, transactions and indexing.
Redis	Caching, short-lived verification state and event/notification coordination.
Queue system	Background AI, voice, reminder and evidence-processing jobs.
Object storage	Scalable evidence/document storage.
Load balancer	Distribute web/API traffic across application instances.
Analytics store	Operational metrics such as doctor utilisation, wait time and no-show rate.

Table 17.1: Planned scaling components

18. Limitations and Future Scope

18.1 Limitations

- AI question quality depends on the model, prompt design and quality of patient input.
- Voice support for regional languages depends on the selected speech provider and its language/accent quality.
- Doctor recommendation quality depends on accurate hospital roster, leave and capacity configuration.

- External AI/voice services may have availability, cost and rate limits.
- Patient deletion must be aligned with legally required medical record retention policies.

18.2 Future Scope

- Mobile application for Android/iOS.
- WhatsApp/SMS/email appointment reminders where approved.
- Hospital-to-hospital referral and transfer workflows.
- More Indian languages and improved Tulu/Kannada speech recognition.
- Analytics for wait time, doctor utilisation, appointment demand and no-shows.
- Clinical-document integration after appropriate privacy and interoperability review.
- More advanced AI summarisation for doctor review with explicit safety controls.

19. Conclusion

The Patient Case-Taking Software using AI addresses a practical gap in hospital intake: patient information, AI-assisted questioning, doctor capacity and appointment scheduling are often treated as separate activities. The proposed platform brings them into one patient-centred workflow for Mangaluru hospitals.

The system begins with verification-code patient access and structured patient details. Symptoms and evidence are attached to a case, and the AI assistant can analyse the information and ask additional questions. Doctor recommendations are based on operational eligibility, and the capacity-aware allocation rule ensures that a full or unavailable doctor does not become a dead end. Patients can select an available check-up date and time, while doctors and hospital authorities receive separate authenticated workflows.

The architecture is intentionally modular. The current Python/Flask and relational-database foundation can be extended with production PostgreSQL, background queues, scalable evidence storage and dedicated AI/voice services. Multilingual support for Tulu, Kannada, English and Hindi provides a patient-facing direction appropriate to the local context.

The central design principle is that AI assists information collection and operational routing but does not replace qualified medical professionals. Security, privacy, auditability and hospital governance remain essential before production deployment.

References

1. Python Software Foundation, Python Documentation, python.org.
2. Flask Documentation, Pallets Projects, flask.palletsprojects.com.
3. SQLite Documentation, sqlite.org/docs.html.
4. PostgreSQL Documentation, postgresql.org/docs/.
5. MDN Web Docs, HTML, CSS and JavaScript documentation, developer.mozilla.org.
6. OWASP Foundation, Application Security Verification Standard and secure web application guidance, owasp.org.
7. World Health Organization, Digital health and health-information guidance, who.int.
8. OpenAI / AI provider documentation, model/API integration and responsible AI guidance, where applicable to the configured implementation.
9. Speech recognition and text-to-speech provider documentation, according to the selected production provider.

10. CareIntake Mangaluru project implementation: supplied patient interface and relational schema used as the project-specific implementation reference.

Appendix A: Glossary

Term	Meaning
AI	Artificial Intelligence used for case analysis, questioning and recommendations.
OTP / Verification Code	A temporary code used to verify a patient's access.
Case	The structured collection of patient details, symptoms, evidence, AI interaction and status.
Evidence	A file or supporting information associated with a patient case.
Doctor Capacity	Configured number of patients/appointments a doctor can accept for a relevant period.
Reassignment	Selecting another eligible doctor when the initial doctor is full/unavailable.
Recommendation Count	Number of suitable doctor/slot recommendations shown; initial documented default is 3.
Tulu	A regional language supported as a planned/configured patient voice option.
Mangaluru / Mangaluru	The geographic focus of the hospital workflow described by the project.
Role-Based Access	Permission model in which patients, doctors and hospital authorities have different capabilities.
Audit Trail	Record of important actions with actor and timestamp.
AI Assistant	Patient-facing component that analyses entered details and asks follow-up questions.

Table A.1: Glossary

Appendix B: Configuration and Project Layout

Item	Description
backend/app.py	Python/Flask application entry point shown in the supplied project.
database/schema.sql	Relational schema containing hospitals, users, doctors, leaves, cases, evidence, appointments, OTPs and audit structures.
database/uploads	Location used for case-linked uploaded evidence in the supplied project layout.
frontend/static/app.js	Client-side application interactions.
frontend/static/style.css	Project styling and layout rules.
frontend/templates/	HTML templates for authentication, patient, doctor, hospital and case workflows.

Table B.1: Project layout and configuration